NEAR One MPC Chain Signatures

Security Assessment

March 6, 2025

Prepared for:
Bowen Wang
NEAR One

Prepared by: **Fredrik Dahlgren, Marc Ilunga, and Jim Miller**

Table of Contents

Table of Contents 1
Project Summary 2
Executive Summary 3
Project Goals 5
Project Targets 6
Project Coverage 7
Automated Testing 9
Summary of Findings 11
Detailed Findings 13
1. Unique IDs from other participants are not validated 13
2. The node uses command-line arguments to pass secrets to the application 16
3. Outdated dependencies with security advisories 18
4. Risk of ciphertext swapping attack on database storage 20
5. Hash function used as key derivation function 22
6. P2P protocol implementation is vulnerable to identity misbinding 24
7. Skipping certificate verification introduces nontrivial modifications to TLS 26
8. Unchecked buffer lengths may enable resource exhaustion 28
9. TLS listener thread does not time out 30
10. Database encryption is vulnerable to nonce reuse 32
11. Insufficient test coverage 34
12. Potential credential persistence in artifacts 36
13. Unpinned external GitHub CI/CD action versions 38
14. Docker release action is vulnerable to cache poisoning 40
15. The P2P networking implementation does not enforce TLS 1.3 42
A. Vulnerability Categories 43
B. Code Quality Findings 45
C. Automated Testing 47
D. The Noise Protocol Framework 49
E. Considerations for Key Refresh and Resharing 52
About Trail of Bits 54
Notices and Remarks 55

Project Summary

Contact Information

The following project manager was associated with this project:

Sam Greenup, Project Manager
sam.greenup@trailofbits.com

The following engineering director was associated with this project:

Jim Miller, Engineering Director, Cryptography
james.miller@trailofbits.com

The following consultants were associated with this project:

Fredrik Dahlgren, Consultant
fredrik.dahlgren@trailofbits.com

Marc Ilunga, Consultant
marc.ilunga@trailofbits.com

Jim Miller, Consultant
james.miller@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
January 9, 2025	Pre-project kickoff call
January 21, 2025	Status update meeting #1
January 28, 2025	Status update meeting #2
February 6, 2025	Delivery of report draft
February 11, 2025	Report readout meeting
March 6, 2025	Delivery of final comprehensive report

Executive Summary

Engagement Overview

NEAR One engaged Trail of Bits to review the security of its MPC protocol node. The MPC protocol node implements a signer running the ECDSA threshold signature scheme Cait-Sith. It supports presignature generation and storage using Beaver triples, as well as additive key generation and presignature rerandomization for online signing.

A team of three consultants conducted the review from January 13 to February 3, 2025, for a total of six engineer-weeks of effort. Our testing efforts focused on ensuring that the node is implemented securely and uses the underlying Cait-Sith library correctly. With full access to source code and documentation, we performed static and dynamic testing of the codebase, using automated and manual processes.

Observations and Impact

Overall, the codebase is well structured and well documented. We found that the implementation uses the Cait-Sith protocol in a secure manner, and we did not identify any issues related to Beaver triple or nonce reuse that could compromise the security of a shared key.

However, we identified several medium-severity issues that could result in hard-to-diagnose protocol disruptions. Some of these issues are due to insufficient data validation (TOB-NEARONE-1, TOB-NEARONE-8). Other issues are caused by misuse of cryptographic primitives and protocols (TOB-NEARONE-6, TOB-NEARONE-4).

We also observed that the unit and integration test coverage is inconsistent across the codebase (TOB-NEARONE-7). The existing tests exercise mostly the happy paths, and there are few tests that exercise the codebase on invalid or unexpected inputs.

Recommendations

Based on the findings identified during the security review, Trail of Bits recommends that NEAR One take the following steps:

- **Remediate the findings disclosed in this report.** These findings should be addressed as part of a direct remediation or any refactor that may occur when addressing other recommendations.
- **Replace the TLS-based peer-to-peer (P2P) protocol with Noise KK.** The current implementation of secure channels is based on TLS with known public keys and dummy certificates. Therefore the implementation is more complex than it needs to be and does not benefit from any of the security guarantees provided by the web PKI. For this reason, we recommend that NEAR One switch to a more lightweight

protocol like Noise KK for the secure channel implementation. Appendix D contains more information on the Noise protocol framework and the Snow library, which is a mature implementation of Noise in Rust.

- **Invest in additional testing.** The codebase has relatively low test coverage and is mostly tested for intended behavior. Additional negative testing will help ensure the robustness of the codebase in adversarial conditions.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

Severity	*Count*
High	0
Medium	7
Low	2
Informational	6
Undetermined	0

CATEGORY BREAKDOWN

Category	*Count*
Auditing and Logging	1
Cryptography	5
Data Exposure	1
Data Validation	2
Denial of Service	2
Patching	1
Testing	3

Project Goals

The review was scoped to provide a security assessment of NEAR One's MPC node implementation. Specifically, we sought to answer the following non-exhaustive list of questions:

- Is the protocol node implemented securely?
- Does the node interface with the underlying Cait-Sith library correctly?
- Are Beaver triples and presignatures managed securely in a manner that prevents reuse?
- Is the additive key generation and presignature rerandomization implemented securely?
- Is the P2P protocol implementation vulnerable to any cryptographic attacks?
- Does the codebase follow modern cryptographic practices?

Project Targets

The engagement involved reviewing and testing the following target.

Near-One/mpc

Repository	https://github.com/Near-One/mpc
Version	`51d5792aafbc47322cf95b0f8e3d0e9da5cd3eec`
Type	Rust
Platform	Multiple

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level assessment goals. Our approaches included the following:

- **Use of the Cait-Sith library:** The implementation of chain signatures relies on the Cait-Sith library for distributed ECDSA signing, key generation, and the generation of Beaver triples and presignatures. The Cait-Sith implementation provides simple and misuse-resistant APIs for users. We reviewed the use of Cait-Sith's subprotocols to check whether they are used safely.

- **Implementation of the P2P network:** Nodes communicate with each other through TLS-based mesh networking. An additional abstraction layer exists to multiplex several MPC tasks atop the mesh network. We reviewed the P2P implementation for common cryptographic vulnerabilities in key exchange protocols and authentication. We also reviewed the implementation's resource management and data validation.

- **Protocol asset management:** The protocol relies on several assets, such as Beaver triples and presignatures. These are generated by background tasks independently of the message to be signed. We reviewed the production and storage of these assets to check if they can be reused by the signing protocol.

- **Custom cryptographic constructions:** The protocol uses certain custom cryptographic constructions, such as rerandomization of presignatures, and a custom key derivation construction. We reviewed them to check if they are sound and provide the expected security properties.

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. The following list outlines the coverage limitations of the assessment and indicates system elements that may warrant further review:

- **Mesh network implementation:** The mesh networking logic is fairly complex due to the multiplexing of concurrent MPC tasks on a single network connection and the less-standard use of TLS. Although we covered the networking implementation, a deeper review of the networking logic would be highly beneficial.

- **Security of the multiplicative rerandomization of presignatures:** Presignature rerandomization is implemented using a construction inspired by the analysis of ECDSA with additive key derivation and presignatures done by Groth and Shoup. However, in the implementation, the randomizers are multiplied by the scalar underlying the presignature rather than additively, as Groth and Shoup

demonstrate. NEAR One conducted a security analysis to prove the security of this modification. We reviewed this modification and the security proof at a high level. However, we would recommend further peer review of the modified rerandomization scheme to ensure that it provides the expected security properties.

Automated Testing

Trail of Bits uses automated techniques to extensively test the security properties of software. We use both open-source static analysis and fuzzing utilities, along with tools developed in-house, to perform automated testing of source code and compiled software.

Test Harness Configuration

We used the following tools in the automated testing phase of this project:

Tool	Description	Policy
`cargo-audit`	A Cargo subcommand that can be used to audit projects dependencies for known vulnerabilities	Appendix C
`cargo-llvm-cov`	A Cargo plugin for generating LLVM source–based code coverage	Appendix C
Clippy	A Rust linter used to catch common mistakes and unidiomatic Rust code	Appendix C
Dylint	An open-source Rust linter developed by Trail of Bits to identify common code quality issues and mistakes in Rust code	Appendix C
Semgrep	An open-source static analysis tool for finding bugs and enforcing code standards when editing or committing code and during build time	Appendix C
`zizmor`	A static analysis tool for GitHub actions	Appendix C

Areas of Focus

Our automated testing and verification work focused on the following:

- General code quality issues and unidiomatic code patterns
- Issues related to error handling and the use of `unwrap` and `expect`
- Extensive use of unsafe code
- Poor unit and integration test coverage

- General issues with dependency management and known vulnerable dependencies
- Common mistakes and security issues in GitHub actions

Test Results

The results of this focused testing are detailed below.

`cargo-audit`

The `cargo-audit` tool identified several outdated dependencies with advisories (TOB-NEARONE-3).

`cargo-llvm-cov`

The coverage report shows areas that are not sufficiently covered by the unit tests (TOB-NEARONE-11).

Clippy and Dylint

Running Clippy in pedantic mode and Dylint identifies several unidiomatic patterns and potential issues related to casting that should be investigated. In appendix C, we describe how to run Clippy in pedantic mode and how to efficiently triage these results using the SARIF file format.

Semgrep

We ran Semgrep Pro on the codebase using both the default configuration and our internal rulesets. This analysis did not identify any security issues in the codebase.

`zizmor`

Running `zizmor` to analyze the GitHub workflows used by the project identifies three low-severity issues (TOB-NEARONE-12, TOB-NEARONE-13, and TOB-NEARONE-14) with how GitHub actions are used.

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Unique IDs from other participants are not validated	Data Validation	Medium
2	The node uses command-line arguments to pass secrets to the application	Data Exposure	Medium
3	Outdated dependencies with security advisories	Patching	Informational
4	Risk of ciphertext swapping attack on database storage	Cryptography	Informational
5	Hash function used as key derivation function	Cryptography	Informational
6	P2P protocol implementation is vulnerable to identity misbinding	Data Validation	Medium
7	Skipping certificate verification introduces nontrivial modifications to TLS	Cryptography	Informational
8	Unchecked buffer lengths may enable resource exhaustion	Denial of Service	Medium
9	TLS listener thread does not time out	Denial of Service	Medium
10	Database encryption is vulnerable to nonce reuse	Cryptography	Medium
11	Insufficient test coverage	Testing	Medium
12	Potential credential persistence in artifacts	Testing	Informational

13	Unpinned external GitHub CI/CD action versions	Auditing and Logging	**Low**
14	Docker release action is vulnerable to cache poisoning	Testing	**Low**
15	The P2P networking implementation does not enforce TLS 1.3	Cryptography	**Informational**

Detailed Findings

1. Unique IDs from other participants are not validated

Severity: **Medium**	Difficulty: **Medium**
Type: Data Validation	Finding ID: TOB-NEARONE-1
Target: `node/src/mpc_client.rs`	

Description

The MPC client does not validate asset IDs from other nodes. This means that nodes can maliciously choose asset IDs to overwrite shared values such as Beaver triples and presignatures owned by other nodes.

Since asset IDs are not validated, a malicious node is free to choose asset IDs belonging to other nodes. If a malicious node chooses an asset ID already used by another node when a Beaver triple or presignature is generated, this will cause other nodes to replace the original asset when the new unowned asset is committed to the database.

```
/// Writes the unowned asset to the db, marking the asset as no longer
/// pending.
pub fn commit(self, value: T) {
    let key = borsh::to_vec(&self.id).unwrap();
    let value_ser = serde_json::to_vec(&value).unwrap();
    let mut update = self.db.update();
    update.put(self.col, &key, &value_ser);
    update
        .commit()
        .expect("Unrecoverable error writing to database");
}
```

Figure 1.1: Honest users will overwrite the original asset in the encrypted database. (`node/src/assets.rs#L407–L417`)

When the owner of the original asset tries to use it to compute a presignature or sign a message, they will take their share of the asset from the owned queue.

```
pub async fn take_owned_with_condition(
    &self,
    cond: impl Fn(&UniqueId, &T) -> bool,
) -> (UniqueId, T) {
    let (id, asset) = self.owned_queue.take_owned_with_condition(cond).await;
    let mut update = self.db.update();
```

```
    update.delete(self.col, &borsh::to_vec(&id).unwrap());
    update
        .commit()
        .expect("Unrecoverable error writing to database");
    (id, asset)
}
```

Figure 1.2: When the owner of the original asset attempts to use it in a protocol instance, she will use her original share from the owned queue. (`node/src/assets.rs#L300–L311`)

But other honest users will read their share of the asset from the database, which will be the asset owned by the malicious node.

```
/// Removes an unowned asset from the storage and returns it. It blocks if
/// the asset is pending, waiting for the computation to complete. It
/// returns an error if we do not have the asset in our database.
pub async fn take_unowned(&self, id: UniqueId) -> anyhow::Result<T> {
    let pending = self.pending_unowned_assets.lock().unwrap().remove(&id);
    if let Some(pending) = pending {
        // We aren't receiving anything, just waiting for the sender to be dropped.
        pending.await.ok();
    }
    let key = borsh::to_vec(&id).unwrap();
    let value_ser = self.db.get(self.col, &key)?.ok_or_else(|| {
        anyhow::anyhow!("Unowned {} not found in the database: {:?}", self.col, id)
    })?;
    let mut update = self.db.update();
    update.delete(self.col, &key);
    update
        .commit()
        .expect("Unrecoverable error writing to database");
    Ok(serde_json::from_slice(&value_ser)?)
}
```

Figure 1.3: Other honest nodes will read the asset directly from the database, resulting in a mismatch. (`node/src/assets.rs#L355–L374`)

This means that the owner of the original ID will use a different asset for the protocol than other honest nodes, which will most likely cause the protocol to fail.

If the asset owned by the malicious node is used before the original asset, all nodes will delete the asset from the database. However, the owner of the original node will still have the value in their owned queue. When the original owner attempts to use this value in a protocol, everyone else will have deleted the asset corresponding to the asset ID and will fail to retrieve the corresponding asset. This will cause the passive listener thread to fail with an error.

It is also possible to corrupt the current asset (and possibly also previously generated assets) by selectively sending different asset IDs to different participants. As an example, if the owner sends an asset ID corresponding to a previously generated asset to some of the

nodes, these nodes will replace their shares of the previous asset with shares of the new asset at the end of the protocol. This would break the expected properties of the previously generated asset.

Ensuring that all honest nodes agree on a unique asset ID is a consensus problem that requires a consensus protocol to solve. Rather than implementing a Byzantine agreement protocol like Practical Byzantine Fault Tolerance (PBFT) to ensure honest nodes agree on the asset ID, we recommend that the NEAR One team investigate whether it is possible to use the existing consensus primitives provided by the NEAR blockchain to solve this problem.

Exploit Scenario

A malicious node operator starts using asset IDs corresponding to already existing assets. This causes protocol failures and crashes that are hard to diagnose, disrupting the system.

Recommendations

Short term, add validation to ensure that an asset ID generated by another node corresponds to the participant ID and does not correspond to a previously reserved asset ID or a previously generated asset.

Long term, investigate whether it is possible to use existing consensus primitives to have the nodes agree on unique IDs. We also recommend that the team simplify the implementation of the distributed asset storage.

2. The node uses command-line arguments to pass secrets to the application

Severity: **Medium**	Difficulty: **High**
Type: Data Exposure	Finding ID: TOB-NEARONE-2
Target: `node/src/cli.rs`	

Description

The command-line interface of the node uses command-line arguments to pass secrets like the root key share and the encryption key for the encrypted database to the application.

```
#[derive(Parser, Debug)]
pub enum Cli {
    /// Runs the node in normal operating mode. A root keyshare must already
    /// exist on disk.
    Start {
        #[arg(long, env("MPC_HOME_DIR"))]
        home_dir: String,
        /// Hex-encoded 16 byte AES key for local storage encryption.
        /// This key should come from a secure secret storage.
        #[arg(env("MPC_SECRET_STORE_KEY"))]
        secret_store_key_hex: String,
        /// Root keyshare, if this is being passed in rather than loaded from disk.
        /// This should be used if the root keyshare is being stored with a secret
        /// manager (such as Google Secret Manager) instead of encrypted on disk.
        /// A bash script should be used to first read the root keyshare from the
        /// secret manager, and then pass it in via this argument.
        /// The root keyshare should be passed in as a JSON string.
        #[arg(env("MPC_ROOT_KEYSHARE"))]
        root_keyshare: Option<String>,
        /// p2p private key for TLS. It must be in the format of "ed25519:...".
        #[arg(env("MPC_P2P_PRIVATE_KEY"))]
        p2p_private_key: SecretKey,
        /// Near account secret key. Signing transactions will only be posted to the
        /// contract if this is specified.
        #[arg(env("MPC_ACCOUNT_SK"))]
        account_secret_key: Option<SecretKey>,
    },
```

Figure 2.1: Secret keys are passed to the application using command-line arguments. (`node/src/cli.rs#L40–L66`)

Since all executed commands are typically stored in the user's shell history, this means that application secrets are recorded here as well. Passing secrets using the command line also makes these secrets available to other users who can run the `ps` command or read the `proc` filesystem on the machine.

Exploit Scenario

Alice runs a NEAR MPC node on her server. Eve, who also has an account on the server, runs `ps`, which lets her read Alice's account secret key and her share of the root key.

Recommendations

Short term, use configuration files to store secrets like the root key share and the database encryption key and pass the filename to the application using the command line instead. Ensure that file permissions are set correctly before reading from and writing to the file.

Long term, allow users to use platform-specific secret managers to store application secrets.

3. Outdated dependencies with security advisories

Severity: **Informational**	Difficulty: **High**
Type: Patching	Finding ID: TOB-NEARONE-3
Target: `node/src/cli.rs`	

Description

The implementation of chain signatures uses outdated dependencies with security advisories. The table below shows the list of outdated dependencies, several of which are unmaintained. The dependencies do not pose a direct threat to the system. However, unmaintained dependencies may hide vulnerabilities that could endanger the system when discovered.

Dependency	Version	ID	Description
`dotenv`	0.15.0	RUSTSEC-2021-0141	Unmaintained
`instant`	0.1.13	RUSTSEC-2024-0384	Unmaintained
`memmap`	0.7.0	RUSTSEC-2020-0077	Unmaintained
`parity-wasm`	0.41.0, 0.42.2	RUSTSEC-2022-0061	Unmaintained
`proc-macro-error`	1.0.4	RUSTSEC-2024-0370	Unmaintained
`wee_alloc`	0.4.5	RUSTSEC-2022-0054	Unmaintained
`lock_api`	0.3.4	RUSTSEC-2020-0070	Some `lock_api` lock guard objects can cause data races
`rustls`	0.23.16	RUSTSEC-2025-0004	Network-reachable panic in `Acceptor::accept`

Table 3.1: Outdated dependencies

Note that the codebase uses tokio-rustls's `TlsAcceptor` API, so is not directly impacted by RUSTSEC-2025-0004.

Recommendations

Short term, update the outdated dependencies to ensure the code is not affected by any potential vulnerability.

Long term, run `cargo-audit` as part of the CI/CD pipeline and ensure that the developers are alerted to any vulnerable dependencies that are detected.

4. Risk of ciphertext swapping attack on database storage

Severity: **Informational**	Difficulty: **High**
Type: Cryptography	Finding ID: TOB-NEARONE-4
Target: `node/src/db.rs`	

Description

The MPC nodes constantly generate preprocessing material (also called assets) such as presignatures and Beaver triples, each associated with a unique ID. These values are then stored encrypted in a key-value store, where the asset ID is used as the key. However, when encrypted, the ciphertexts are not cryptographically tied to the ID they belong to, allowing an attacker to swap ciphertext values without detection.

Figure 4.1 shows how a value is encrypted and then stored in the database.

```rust
/// Puts a key-value pair into the database, overwriting if the key
/// already exists. Encrypts the value before persisting it.
pub fn put(&mut self, col: DBCol, key: &[u8], value: &[u8]) {
    let value = encrypt(&self.db.cipher, value);
    self.batch.put_cf(&self.db.cf_handle(col), key, &value);
}
```

Figure 4.1: The database encryption code does not tie the asset ID to the encrypted asset. (`node/src/db.rs#L126–L131`)

Conversely, figure 4.2 shows how a value is retrieved from the database.

```rust
/// Gets the specified value from the database.
/// The value is decrypted before being returned.
pub fn get(&self, col: DBCol, key: &[u8]) -> anyhow::Result<Option<Vec<u8>>> {
    let value = self.db.get_cf(&self.cf_handle(col), key)?;
    value.map(|v| decrypt(&self.cipher, &v)).transpose()
}
```

Figure 4.2: A value is retrieved from the database and decrypted before it is returned. (`node/src/db.rs#L78–L83`)

Figure 4.2 shows that when a key-value pair is retrieved, there is no mechanism that ties the ciphertext to the purported key it was initially stored with. As a consequence, an attacker who can modify the database is able to swap ciphertexts without being detected.

The consequences of an attacker swapping database values range from disruptions to the service to allowing reuse of preprocessing material. If an attacker swaps two ciphertexts

(two Beaver triples, for example), it is very likely that the protocol will not succeed. Furthermore, this issue will be hard to diagnose.

Additionally, in a more security-critical scenario, an attacker may swap ciphertexts with previously used presignatures, which would endanger the security of the MPC protocol. If such an attacker could corrupt every node in the protocol, although unlikely, it would result in the compromise of the group's secret key.

Exploit Scenario

An attacker gains writing privileges to the database. They modify the database to swap ciphertexts. The attack goes undetected and causes a protocol failure.

Recommendations

Short term, include the unique ID as associated data passed onto the authenticated encryption scheme.

Long term, consider computing a MAC over the AEAD tags when the node shuts down and then verifying the MAC on start-up. This would prevent malicious local users from reintroducing already used key-value pairs into the database. Also, consider having nodes maintain state to prevent replays of previously used database assets.

5. Hash function used as key derivation function

Severity: **Informational**	Difficulty: **High**
Type: Cryptography	Finding ID: TOB-NEARONE-5
Target: `node/src/hkdf.rs`	

Description

The chain signature implementation uses additive key derivation to derive subkeys from the main signing key. However, the randomizer is constructed using a custom hash-based key derivation function (KDF) rather than a dedicated KDF.

In the NEAR system, a set of nodes performs ECDSA signatures using a secure multiparty protocol, Cait-Sith. Given the main public key $X = xG$, a user may obtain an additional subkey by specifying a derivation path. To derive a subkey, first, a tweak e is derived by hashing the user's account ID and the derivation path. The derived subkey is the pair $(x + e, X + eG)$. The private key is not explicitly constructed but is shared among nodes.

In the implementation, SHA-3 is used to derive the tweak, as shown in figure 5.1.

```
// TODO: Modify the following function and use instead hkdf.
// WARNING: DO NOT change anything before making sure that the legacy secret/public
keys are also changed
// and stored signatures could still be verified.
const TWEAK_DERIVATION_PREFIX: &str = "near-mpc-recovery v0.1.0 epsilon
derivation:";

pub fn derive_tweak(predecessor_id: &AccountId, path: &str) -> Scalar {
    // ',' is ACCOUNT_DATA_SEPARATOR from nearcore that indicate the end
    // of the accound id in the trie key. We reuse the same constant to
    // indicate the end of the account id in derivation path.
    // Do not reuse this hash function on anything that isn't an account
    // ID or it'll be vunerable to Hash Melleability/extention attacks.
    let derivation_path = format!("{TWEAK_DERIVATION_PREFIX}{},{}", predecessor_id,
path);
    let mut hasher = Sha3_256::new();
    hasher.update(derivation_path);
    let hash: [u8; 32] = hasher.finalize().into();
    Scalar::from_bytes(hash).expect(
        "Expected hash of derived key to be in the
        field of size 2^256 - 2^32 - 2^9 - 2^8 - 2^7 - 2^6 - 2^4 - 1 ",
    )
}
```

Figure 5.1: The node uses SHA-3 to derive the tweak. (`node/src/hkdf.rs#L89–L108`)

Groth and Shoup proved the security of additive key derivation using hashing in the random oracle model. Although SHA-3 behaves like a random oracle in the sense of indifferentiability, a dedicated KDF, like HKDF, has more practical advantages over using a hash function. More precisely, not all hash functions behave like random oracles in general or for the purpose of key derivation in particular. As an example, swapping the hash function for another function like SHA-256 would be problematic.

Recommendations

Short term, use a dedicated KDF like HKDF to derive the tweak. When using HKDF, the salt must not be attacker-controlled, as using attacker-controlled input in the salt input can lead to collisions. A fixed-length value for the salt is sufficient.

Long term, review all cryptographic primitives used in the NEAR system to ensure that they are used conservatively. Consider systematizing domain separation for key derivation and hashing implementations. For example, have each KDF take the hash of a unique string as the salt for domain separation. Hash functions should then use the hash of the unique string as a prefix to the input for domain separation.

6. P2P protocol implementation is vulnerable to identity misbinding

Severity: **Medium**	Difficulty: **Medium**
Type: Data Validation	Finding ID: TOB-NEARONE-6
Target: `node/src/indexer/participants.rs`, `node/src/p2p.rs`	

Description

Participant metadata like the peer ID and P2P public key of the participant is stored on-chain. This data is read by the indexer and is used to set up P2P encrypted channels between all participants in the group. When a remote peer initiates a TLS connection to the node, the P2P public key from the client certificate is used to identify the remote node.

```
let Ok(public_key) = cert.1.public_key().parsed() else {
    anyhow::bail!("Connection with invalid public key");
};
// The library doesn't recognize ED25519 keys, but that's fine, we'll compare the
// raw bytes directly.
let PublicKey::Unknown(public_key_data) = public_key else {
    anyhow::bail!(
        "Connection with unexpected public key type: {:?}",
        public_key
    );
};
let public_key = near_crypto::ED25519PublicKey(
    public_key_data
        .try_into()
        .context("Connection with public key of unexpected length")?,
);
let Some(peer_id) = participant_identities
    .key_to_participant_id
    .get(&near_crypto::PublicKey::ED25519(public_key))
else {
    anyhow::bail!("Connection with unknown public key");
};
```

Figure 6.1: The public key is used to identify the connecting peer in the `verify_peer_identity` function. (`node/src/p2p.rs#L562–L583`)

If a malicious participant Malory chooses the same P2P public key as another participant Alice, this will not be detected by the node and Alice's entry in the `key_to_participant_id` mapping will be overwritten by Malory's when the mapping is created. This means that all connections originating from Alice will be misidentified as coming from Malory.

```
for participant in &config.participants.participants {
    participant_ids.push(participant.id);
    participant_identities
        .key_to_participant_id
        .insert(participant.p2p_public_key.clone(), participant.id);
}
```

Figure 6.2: The implementation does not ensure that all public keys are distinct when the `key_to_participant_id` mapping is created. (`node/src/p2p.rs#L456–L461`)

Since Alice controls the private key corresponding to the shared public key, she will be able to establish TLS connections to other nodes in the network. However, other nodes will believe that they are communicating with Malory and will misattribute all messages to her.

We could not identify an immediate way to exploit this situation, as malicious behavior will likely be detected by the underlying MPC protocol. However, in the general case, this type of identity misbinding can often lead to serious security issues, often allowing malicious users to access privileged resources belonging to other users. The security proof for Cait-Sith also depends on the assumption that participants are connected using encrypted and authenticated channels, which is no longer true here.

Exploit Scenario

Malory, a malicious node operator, registers the same public key as another node operator Alice. When the on-chain participant data is parsed by the nodes, Alice's entry in the `key_to_participant_id` mapping is overwritten by Malory's, causing Alice's traffic to be misattributed to Malory. This causes failures that are hard to diagnose in the underlying MPC protocol.

Recommendations

Short term, implement checks to ensure that all P2P public keys are unique across the entire group.

Long term, consider implementing a proof of possession for public keys posted on-chain by the nodes. For the P2P public key, the issue could also be mitigated by adding the peer ID as the subject name of the certificate. Since each participant signs and generates a MAC for the entire transcript (including certificates) in TLS 1.3, this would also serve as a proof of possession. The peer ID of the remote node can then be retrieved from the certificate.

In general, we recommend that the team review the use of maps across the codebase to understand the risk of overwrite attacks in the case when the key is controlled by a remote node.

7. Skipping certificate verification introduces nontrivial modifications to TLS

Severity: **Informational**	Difficulty: **High**
Type: Cryptography	Finding ID: TOB-NEARONE-7
Target: `node/src/p2p.rs`	

Description

The P2P server requires client authentication. However, it does not verify the client certificate; it relies on custom logic to verify the validity of the public key presented in the certificate. Support for this custom logic is enabled through nontrivial modifications to the verification logic of TLS, including custom data structures to verify certificates. For example, a custom verifier, `DummyClientCertVerifier`, had to be implemented. These changes make the code more complex than needed and increase the maintenance burden.

Figure 7.1 shows the server configuration used to run TLS. Client certificate verification is done through the `DummyClientCertVerifier`.

```
let server_config = rustls::ServerConfig::builder()
    .with_client_cert_verifier(Arc::new(DummyClientCertVerifier))
    .with_single_cert(vec![p2p_cert.der().clone()], p2p_key_der.clone_key())?;
```

Figure 7.1: The server uses a custom certificate verifier that always accepts the incoming client certificate. (`node/src/p2p.rs#L421–L423`)

The `DummyClientCertVerifier` implementation performs no certificate validation and simply accepts the incoming certificate, as figure 7.2 shows.

```
impl ClientCertVerifier for DummyClientCertVerifier {
    fn root_hint_subjects(&self) -> &[rustls::DistinguishedName] {
        &[]
    }

    fn verify_client_cert(
        &self,
        _end_entity: &CertificateDer<'_>,
        _intermediates: &[CertificateDer<'_>],
        _now: rustls::pki_types::UnixTime,
    ) -> std::result::Result<
        rustls::server::danger::ClientCertVerified,
        rustls::Error
    > {
        Ok(rustls::server::danger::ClientCertVerified::assertion())
    }
```

```rust
    fn verify_tls12_signature(
        &self,
        _message: &[u8],
        _cert: &CertificateDer<'_>,
        _dss: &rustls::DigitallySignedStruct,
    ) -> std::result::Result<
        rustls::client::danger::HandshakeSignatureValid,
        rustls::Error
    > {
        Ok(rustls::client::danger::HandshakeSignatureValid::assertion())
    }

    fn verify_tls13_signature(
        &self,
        _message: &[u8],
        _cert: &CertificateDer<'_>,
        _dss: &rustls::DigitallySignedStruct,
    ) -> std::result::Result<
        rustls::client::danger::HandshakeSignatureValid,
        rustls::Error
    > {
        Ok(rustls::client::danger::HandshakeSignatureValid::assertion())
    }

    fn supported_verify_schemes(&self) -> Vec<rustls::SignatureScheme> {
        vec![rustls::SignatureScheme::ED25519]
    }
}
```

Figure 7.2: The dummy verifier simply accepts all incoming certificates. (`node/src/p2p.rs#L55–L90`)

TLS 1.3 supports mutual authentication, allowing the client and the server to authenticate and verify each other's certificates. The Rust crate `rustls` offers a `WebPkiClientVerifier` object, which verifies certificates based on a root certificate store. In the P2P implementation, the client configuration already uses a root certificate store. The server configuration may also simply use the same root store.

Recommendations

Short term, implement client verification as prescribed by TLS. This can be achieved by using a server configuration with client verification built with a `WebPkiClientVerifier` that uses the same certificate root store the client configuration uses.

Long term, consider moving to a simpler secure communication implementation, as described in appendix D. Ensure uses of cryptographic protocols remain as close as possible to their intended use cases.

8. Unchecked buffer lengths may enable resource exhaustion

Severity: **Medium**	Difficulty: **High**
Type: Denial of Service	Finding ID: TOB-NEARONE-8
Target: `node/src/p2p.rs`	

Description

Each MPC node runs a TLS server that continuously handles incoming TLS connections. The server allocates a byte buffer whose length is determined by the first bytes of certain incoming data-carrying packets. An attacker can set the packet length maliciously to exhaust the resources of a target server. Furthermore, a malicious client may create multiple connections to amplify resource exhaustion.

The server reads incoming messages, which are then parsed and dispatched to the appropriate processing, assuming the message comes from a data-carrying packet. The server reads the first four bytes and then creates a byte vector of the prescribed length, as figure 8.1 shows.

To exhaust a server's memory, a malicious client can set the length to an arbitrarily large value. However, successful resource exhaustion will require the adversary to send enough data to fill memory. The attack is also constrained to a short time window that depends on the current network task. However, depending on the platform, Rust will not commit to this memory unless it is written to. Furthermore, each network operation is part of a network "task," which should time out after the preconfigured amount of time.

Nevertheless, the attack is rather straightforward for an attacker to implement. Furthermore, the time window limitation can be overcome by two factors. First, fairly high bandwidths can be guaranteed between instances in a cloud deployment. For example, well-optimized N1 machines can achieve up to more than 1 Gbps bandwidths, as described in this guide. Therefore, a maximum memory allocation of 2^{32} bytes (around 4 GB) can be easily filled in less than 60 seconds. Secondly, the server does not enforce a single TCP stream from a given client. This means that the attack can be amplified by creating several connections, each forcing the server to dedicate 2^{32} bytes, which the malicious client would then fill to exhaust resources. A group of malicious nodes can further amplify the effect of this attack.

```
loop {
    let tag = stream.read_u8().await?;
    match tag {
```

```
        0 => {
            // Ping
            received_bytes += 1;
        }
        1 => {
            let mut len_buf = [0u8; 4];
            stream.read_exact(&mut len_buf).await?;
            let len = u32::from_be_bytes(len_buf) as usize;
            let mut buf = vec![0u8; len];
            stream.read_exact(&mut buf).await?;
            let message = MpcPeerMessage {
                from: peer_id,
                message: MpcMessage::try_from_slice(&buf)?,
            };
            message_sender.send(message).await?;
            received_bytes += 5 + len as u64;
        }
        _ => {
            anyhow::bail!("Invalid tag");
        }
    }
    tracking::set_progress(&format!(
        "Received {} bytes from {}",
        received_bytes, peer_id
    ));
}
```

Figure 8.1: TLS buffer lengths are not validated by the node. (`node/src/p2p.rs#L502–L530`)

The severity of this issue is rated as medium because it impacts the availability of the group. On the other hand, to carry out the attack, the sender needs to be part of the group, which implies a high level of privilege. For this reason, the difficulty is rated as high.

Exploit Scenario

A malicious node wants to delay the signature of a certain transaction. They open several connections to other nodes in the network and send large amounts of data to exhaust their memory.

Recommendations

Short term, add validation of the length of messages that are read from the TCP stream to ensure it does not exceed a predefined maximum message length.

Long term, review the entire codebase to ensure all potentially attacker-controlled inputs are validated before use.

9. TLS listener thread does not time out

Severity: **Medium**	Difficulty: **High**
Type: Denial of Service	Finding ID: TOB-NEARONE-9
Target: `node/src/p2p.rs`	

Description

The TLS listener thread creates a new task for each new incoming connection. The task reads a 4-byte length *L* and then waits for a message of length *L* from the network. If the remote node sends less than *L* bytes, the thread will hang, waiting for the message to complete. This ties up local resources and can be used to exhaust processing and memory resources of the node.

```
loop {
    let tag = stream.read_u8().await?;
    match tag {
        0 => {
            // Ping
            received_bytes += 1;
        }
        1 => {
            let mut len_buf = [0u8; 4];
            stream.read_exact(&mut len_buf).await?;
            let len = u32::from_be_bytes(len_buf) as usize;
            let mut buf = vec![0u8; len];
            stream.read_exact(&mut buf).await?;
            let message = MpcPeerMessage {
                from: peer_id,
                message: MpcMessage::try_from_slice(&buf)?,
            };
            message_sender.send(message).await?;
            received_bytes += 5 + len as u64;
        }
        _ => {
            anyhow::bail!("Invalid tag");
        }
    }
    tracking::set_progress(&format!(
        "Received {} bytes from {}",
        received_bytes, peer_id
    ));
}
```

Figure 9.1: Since the listener thread does not time out, the node can be left waiting for incoming data indefinitely. (`node/src/p2p.rs#L512–L540`)

We note that the parent task controlling the execution of the entire MPC protocol instance will time out eventually. The protocol timeout is part of the node configuration and is typically on the order of 60 seconds. However, there is no log message identifying the misbehaving node, which makes troubleshooting this type of issue unnecessarily difficult.

The severity of this issue is rated as medium because it could impact the availability of the group. On the other hand, to carry out the attack, the sender needs to be part of the group, which implies a high level of privilege. For this reason, the difficulty is rated as high.

Exploit Scenario

A malfunctioning MPC node drops outgoing packets, which causes other nodes to hang while waiting for incoming data. This causes intermittent protocol failures that are hard for the NEAR One team to diagnose.

Recommendations

Short term, use a preconfigured timeout to ensure that the TLS listener thread is not left waiting for data from malfunctioning or malicious nodes.

Long term, extend the use of logging to make it easier for node operators to investigate unexpected behavior. Ensure that log messages originating from potentially malicious behavior always identify the corresponding remote node.

10. Database encryption is vulnerable to nonce reuse

Severity: **Medium**	Difficulty: **High**
Type: Cryptography	Finding ID: TOB-NEARONE-10
Target: `node/src/db.rs`	

Description

The asset database uses AES-GCM to encrypt stored assets such as Beaver triples and presignatures. The encryption key is provided by the user when the node is initialized and is never rotated. AES-GCM nonces are generated pseudorandomly in each call to `encrypt`.

```
/// Encrypts a single value with AES-GCM. This encryption is randomized.
pub fn encrypt(cipher: &Aes128Gcm, plaintext: &[u8]) -> Vec<u8> {
    let nonce = aes_gcm::Aes128Gcm::generate_nonce(&mut rand::thread_rng());
    let ciphertext = cipher.encrypt(&nonce, plaintext).unwrap();
    [nonce.as_slice(), &ciphertext].concat()
}
```

Figure 10.1: Database values are encrypted using AES-GCM with a random 96-bit nonce. (`node/src/db.rs#L43–L48`)

The AES-GCM nonce is 96 bits, which means that the probability of a nonce collision after 2^n calls to `encrypt` is $2^{-(96-n)/2}$. In particular, if 2^{32} assets are stored in the database over the entire lifetime of the node, the probability of a nonce collision is 2^{-32}. If an AES-GCM nonce is ever reused, an attacker with access to the encrypted database could recover the authentication key with high probability, which would allow her to forge authentication tags and completely break the authenticity guarantees of the cipher for the key in question.

For this reason, NIST SP 800-38D recommends restricting the number of invocations of AES-GCM for a given key to at most 2^{32} calls when a pseudorandom nonce is used. Since the MPC node is expected to cache around 1,000,000 (~2^{20}) Beaver triples, which in turn corresponds to 500,000 (~2^{19}) presignatures, this means that long-running nodes may risk hitting the 2^{32} bound over time.

Exploit Scenario

An attacker gains continuous access to the encrypted database of a single MPC node. She discovers an instance where the same nonce is reused to encrypt two different values, which allows her to recover the authentication key and forge ciphertexts stored in the database. Since all values are encrypted using AES-CTR, this allows her to effectively edit values in the database that she could use to leak secrets during protocol execution.

Recommendations

Short term, switch to a different database encryption scheme with extended nonces, like XChaCha20-Poly1305, which is better suited for long-lived keys.

Long term, consider implementing functionality that allows the user to rotate the database encryption key. Track the number of calls to `encrypt` over the entire lifetime of the key and alert the user when the encryption key should be rotated to ensure that the risk of nonce reuse remains negligible.

References

- NIST SP 800-38D
- RustCrypto documentation for XChaCha20-Poly1305

11. Insufficient test coverage

Severity: **Medium**	Difficulty: **N/A**
Type: Testing	Finding ID: TOB-NEARONE-11
Target: node	

Description

The codebase includes both unit and integration tests, but the average line coverage is only 75%. Furthermore, several important files, like the implementation of the signing functionality, have very limited coverage, and other files are not tested at all.

Filename	Function Coverage	Line Coverage	Region Coverage
assets.rs	93.44% (57/61)	96.99% (613/632)	87.70% (214/244)
background.rs	100.00% (6/6)	100.00% (26/26)	100.00% (6/6)
cli.rs	32.00% (8/25)	66.67% (218/327)	55.17% (96/174)
config.rs	76.47% (13/17)	85.29% (58/68)	63.49% (40/63)
db.rs	100.00% (20/20)	99.31% (144/145)	83.67% (82/98)
hkdf.rs	83.33% (10/12)	88.27% (143/162)	91.18% (31/34)
indexer/configs.rs	0.00% (0/8)	0.00% (0/42)	0.00% (0/15)
indexer/handler.rs	0.00% (0/10)	0.00% (0/132)	0.00% (0/94)
indexer/lib.rs	0.00% (0/6)	0.00% (0/48)	0.00% (0/29)
indexer/mod.rs	0.00% (0/3)	0.00% (0/22)	0.00% (0/3)
indexer/participants.rs	75.00% (12/16)	79.13% (163/206)	60.92% (53/87)
indexer/response.rs	26.67% (4/15)	40.87% (103/252)	27.36% (29/106)
indexer/stats.rs	0.00% (0/5)	0.00% (0/53)	0.00% (0/30)
indexer/transaction.rs	0.00% (0/3)	0.00% (0/44)	0.00% (0/8)
key_generation.rs	90.00% (18/20)	94.51% (155/164)	76.67% (69/90)
main.rs	0.00% (0/3)	0.00% (0/20)	0.00% (0/25)
mpc_client.rs	90.91% (10/11)	53.70% (138/257)	32.99% (32/97)
network.rs	92.00% (46/50)	94.91% (373/393)	81.18% (151/186)
p2p.rs	92.86% (52/56)	94.18% (712/756)	83.06% (309/372)
primitives.rs	60.00% (15/25)	55.71% (39/70)	47.06% (24/51)
protocol.rs	100.00% (16/16)	99.45% (181/182)	97.37% (74/76)
sign.rs	30.77% (4/13)	25.50% (51/200)	19.35% (12/62)
sign_request.rs	25.00% (2/8)	33.87% (21/62)	10.26% (4/39)
tracing.rs	100.00% (1/1)	100.00% (6/6)	100.00% (1/1)
tracking.rs	96.77% (30/31)	97.27% (214/220)	91.03% (71/78)
triple.rs	100.00% (17/17)	96.37% (239/248)	84.38% (81/96)
web.rs	50.00% (2/4)	52.94% (9/17)	50.00% (2/4)
web_test.rs	73.68% (14/19)	80.99% (98/121)	67.12% (49/73)
Totals	**74.22% (357/481)**	**75.98% (3704/4875)**	**63.81% (1430/2241)**

Figure 11.1: Test coverage result obtained with `cargo-llvm-cov` (excludes networking experiment test in `research.rs`)

Additionally, the existing test suite mainly exercises positive code paths, testing for the expected functionality. There are few tests for unexpected behavior or invalid inputs. For example, the tests implemented in `p2p.rs` test only for robustness when a participant faces networking issues. The code is not tested for misbehaving network clients sending unexpected or invalid data. This type of negative testing helps increase confidence in the robustness of the codebase when deployed in adversarial environments.

Recommendations

Short term, increase the code coverage, targeting at least 90% as a minimal baseline. Implement additional negative tests to exercise negative paths and ensure the robustness of the codebase. Use tools like `cargo-llvm-cov` to monitor testing coverage regularly.

Long term, monitor test coverage as part of the CI/CD process to ensure that new components are covered by the test suite.

12. Potential credential persistence in artifacts

Severity: **Informational**	Difficulty: **High**
Type: Testing	Finding ID: TOB-NEARONE-12
Target: `.github/workflows/ci.yml`, `.github/workflows/docker_release.yml`	

Description

The default behavior of the `actions/checkout` GitHub action is to persist credentials, which means that the GitHub token is written to the local repository directory. This allows the action to execute Git commands that require authentication. The credentials are stored in `.git/config` files in the local repository directory, where they could be inadvertently included in workflow artifacts or be accessed by subsequent workflow steps.

```yaml
- name: Checkout repository
  uses: actions/checkout@v4
  with:
    ref: ${{ github.event.inputs.build-ref }}
```

Figure 12.1: The `actions/checkout` GitHub action persists GitHub credentials by default. (`.github/workflows/docker_release.yml#L24–L27`)

Looking at the Dockerfile used to build the Docker container for the application—`deployment/Dockerfile-gcp`—we see that only the target executable and the `deployment/gcp-start.sh` script are present in the final container (figure 12.2).

```
FROM chef AS builder
COPY --from=planner /app/recipe.json recipe.json
COPY .git .git
COPY .gitmodules .gitmodules

# ...

FROM google/cloud-sdk:debian_component_based AS runtime
RUN apt-get update -y \
    && apt-get install -y --no-install-recommends openssl ca-certificates
WORKDIR /app
COPY --from=builder /app/target/release/mpc-node mpc-node
COPY deployment/gcp-start.sh /app/gcp-start.sh
```

Figure 12.2: The `.git` directory is copied into the container. (`deployment/Dockerfile-gcp#L30–L35`)

However, the `.git` subdirectory is copied into the container in earlier stages, which means that earlier layers may contain GitHub credentials. Since the `docker-image-build` job

defined in `docker_release.yml` uses caching, these layers may be persisted in the GitHub actions cache and may be readable from forks of the repository.

```yaml
jobs:
  docker-image-build:
    name: "Build and push MPC Docker image"
    runs-on: ubuntu-22.04-8core
    permissions:
      contents: read

    steps:
      # ...
      - name: Build and push MPC Docker image to Docker Hub
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          file: deployment/Dockerfile-gcp
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

Figure 12.3: Docker container layers may be persisted to the GitHub actions cache. (`.github/workflows/docker_release.yml#L16–L54`)

However, we note that this is currently not a security issue since the GitHub token used by the workflow has only read permissions to the repository. (This is also true for the token used by the CI workflow.)

Exploit Scenario

A refactor inadvertently removes the `permissions` parameter from the Docker release workflow. This causes the GitHub token to inherit the default permissions from the organization. Since the Docker container layers are cached by the release workflow, the GitHub token is persisted in the GitHub actions cache. An attacker forks the repository, which allows them to read the cache through a pull request to the main repository. They obtain the potentially overly scoped credential and use this credential to further compromise the repository.

Recommendations

Short term, add `persist-credentials: false` to checkout actions that do not require privileged Git operations.

Long term, add `actionlint`, `poutine`, and `zizmor` to the CI/CD pipeline.

References

- ArtiPACKED: Hacking Giants Through a Race Condition in GitHub Actions Artifacts
- `zizmor` ArtiPACKED documentation

13. Unpinned external GitHub CI/CD action versions

Severity: **Low**	Difficulty: **High**
Type: Auditing and Logging	Finding ID: TOB-NEARONE-13
Target: `.github/workflows/ci.yml`, `.github/workflows/docker_release.yml`	

Description

Several GitHub Actions workflows in the `mpc` repository use third-party actions pinned to a tag or branch name instead of a full commit SHA as recommended by GitHub. This configuration enables repository owners to silently modify the actions. A malicious actor could use this ability to tamper with an application release or leak secrets.

The following actions are owned by GitHub organizations or individuals that are not affiliated directly with NEAR One:

- `Swatinem/rust-cache@v2`
- `docker/login-action@v3`
- `docker/setup-buildx-action@v3`
- `docker/build-push-action@v6`

```
- name: Cache Rust dependencies
  uses: Swatinem/rust-cache@v2
  with:
    cache-on-failure: true
    cache-all-crates: true
```

Figure 13.1:GitHub workflows depend on a number of third-party actions that are pinned only to a tag. (`.github/workflows/ci.yml#L27–L31`)

Exploit Scenario

An attacker gains unauthorized access to the account of a GitHub action owner. The attacker manipulates the action's code to steal GitHub credentials and compromise the repository.

Recommendations

Short term, pin each third-party action to a specific full-length commit SHA, as recommended by GitHub. Additionally, configure Dependabot to update the actions' commit SHAs after reviewing their available updates.

Long term, add `zizmor` to the CI/CD pipeline.

14. Docker release action is vulnerable to cache poisoning

Severity: **Low**	Difficulty: **High**
Type: Testing	Finding ID: TOB-NEARONE-14
Target: `.github/workflows/ci.yml`, `.github/workflows/docker_release.yml`	

Description

The Docker release GitHub workflow uses caching to store the `buildx` binary as well as intermediate build artifacts to the GitHub actions cache. An attacker who gains write access to the GitHub actions cache (e.g., through a valid GitHub token) can poison the cache and compromise the build process to maliciously alter build artifacts.

```yaml
jobs:
  docker-image-build:
    name: "Build and push MPC Docker image"
    runs-on: ubuntu-22.04-8core
    permissions:
      contents: read

    steps:
      # ...
      - name: Set up Docker Buildx                          # Uses caching
        uses: docker/setup-buildx-action@v3

      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Build and push MPC Docker image to Docker Hub   # Uses caching
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          file: deployment/Dockerfile-gcp
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

Figure 14.1: A number of the GitHub actions in the Docker release workflow depend on caching. (`.github/workflows/docker_release.yml#L16–L54`)

Exploit Scenario

An attacker gains unauthorized access to a GitHub token with write permissions to the GitHub actions cache. She uses this to poison the cache to inject a malicious `buildx`

binary. When the Docker container is built, the final image is modified to download and run attacker-controlled software. Since the codebase is not tampered with directly, the attack goes unnoticed by the NEAR One team.

Recommendations

Short term, in general, avoid using previously cached CI state within workflows intended to publish build artifacts such as Docker containers.

Long term, add `zizmor` to the CI/CD pipeline to detect GitHub action–related issues.

15. The P2P networking implementation does not enforce TLS 1.3

Severity: **Informational**	Difficulty: **High**
Type: Cryptography	Finding ID: TOB-NEARONE-15
Target: `node/src/p2p.rs`	

Description

The TLS configuration used by the P2P networking implementation does not currently enforce TLS 1.3. As Rustls supports TLS 1.2 by default, a malicious node could force connecting nodes to use TLS 1.2 rather than TLS 1.3, which could make the connection vulnerable to further attacks.

```
// As the server, we do not verify the client's certificate, but we still need
// a custom verifier or else the certificate will not even be propagated to us
// when we handle the connection. Later we'll check that the client provided a
// valid public key in the certificate.
let server_config = rustls::ServerConfig::builder()
    .with_client_cert_verifier(Arc::new(DummyClientCertVerifier))
    .with_single_cert(vec![p2p_cert.der().clone()], p2p_key_der.clone_key())?;
// As a client, we verify that the server has a valid certificate signed by the
// dummy issuer (this is required by rustls). When making the connection we also
// check that the server has the right public key.
let client_config = rustls::ClientConfig::builder()
    .with_root_certificates(root_cert_store)
    .with_client_auth_cert(vec![p2p_cert.der().clone()], p2p_key_der.clone_key())?;
```

Figure 15.1: The `configure_tls` function does not enforce TLS 1.3. (`node/src/p2p.rs#L427–L439`)

However, it is important to note that as Rustls supports TLS 1.3, honest nodes will always negotiate TLS 1.3 rather than 1.2. This means that protocol downgrades could only affect connections where at least one endpoint is running a custom version of the NEAR One MPC node.

Recommendations

Short term, use the `ConfigBuilder::with_protocol_versions` method to enforce TLS 1.3 when the client and server configurations are set up.

Long term, ensure that the codebase is always using the highest possible version of any cryptographic protocols that it relies on. Deprecate earlier versions of these protocols as new versions are standardized and become available.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity, and difficulty levels used in this document.

Vulnerability Categories	
Category	**Description**
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	**Description**
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	**Description**
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Quality Findings

- **Functions with too many lines.** The `run` functions in `node/src/cli.rs` and `node/src/mpc_client.rs` have long bodies. Functions with many lines hinder the readability of the codebase and may also complicate unit testing.

- **Inconsistent struct instantiation.** When the `StartResponse` structure is instantiated by the `Cli::run` method, the fields are ordered differently than in the structure declaration. Although this pattern is allowed in Rust, it decreases the readability of the codebase.

```rust
pub struct StartResponse {
    pub mpc_handle: Handle<Result<(), anyhow::Error>>,
    pub indexer_handle: Option<JoinHandle<()>>,
}
```

Figure B.1: The `StartResponse` struct (`node/src/cli.rs#L123–L126`)

```rust
Ok(Some(StartResponse {
    indexer_handle,
    mpc_handle,
}))
```

Figure B.2: Instantiation with a different field order (`node/src/cli.rs#L340`)

- **Lack of logging in the `propose_update` function in `chain-signatures/contract/src/lib.rs`.** The function does not log any information using `log!`. The other public functions in `chain-signatures/contract/src/lib.rs` all use `log!` to log important information.

- **Use of magic numbers in `decrypt` function.** The `decrypt` function in `node/src/db.rs` defines a `const` value `NONCE_LEN` and sets it to 12. However, despite having this `const` value, the function uses hard-coded magic numbers of 12 for both the nonce and `ciphertext` variables.

- **Commented-out code.** The file `p2p.rs` contains code that is commented out.

```rust
// let endpoint_for_listener = server.clone();
```

Figure B.3: Line of code that is commented out (`node/src/p2p.rs#L483`)

- **Unnamed constants in `network.rs` and other files.** The buffer size and LRU cache capacity used to run a new `MeshNetworkClient` are hard-coded numbers. Hard-coded numbers without context make the codebase hard to understand and maintain. Several occurrences of magic numbers can be found in other files.

```
   deleted_channels:
Arc::new(Mutex::new(LruCache::new(10000.try_into().unwrap()))),
});
let (new_channel_sender, new_channel_receiver) = mpsc::channel(1000);
```

Figure B.4: Magic numbers in `network.rs` (`node/src/network.rs#L217–L219`)

- **Explicit `into_iter` loop.** The use of `into_iter` in the following code snippet is unnecessary and can be removed to improve readability.

```
for (p, messages) in messages_to_send.into_iter() {
    if messages.is_empty() {
        continue;
    }
    counters.queue_send(p, messages.len());
    // There's a chance this sending can fail, because the sending task can
    // return early if the connection to some other participant is broken. In
    // that case, the computation task should also just fail.
    queue_senders.get(&p).unwrap().send(messages)?;
}
```

Figure B.5: Iterating over messages with an explicit call to `into_iter` (`node/src/protocol.rs#L113–L122`)

C. Automated Testing

This section describes the setup of the automated analysis tools used during this audit.

cargo-audit

The `cargo-audit` Cargo plugin identifies known vulnerable dependencies in Rust projects. It can be installed using `cargo install cargo-audit`. To run the tool, run `cargo audit` in the crate root directory.

cargo-llvm-cov

The `cargo-llvm-cov` Cargo plugin is used to generate LLVM source-based code coverage data. The plugin can be installed via the command `cargo install cargo-llvm-cov`. To run the tool, run the command `cargo llvm-cov` in the crate root directory.

Clippy

The Rust linter Clippy can be installed using `rustup` by running the command `rustup component add clippy`. Invoking `cargo clippy -- -W clippy::pedantic` in the root directory of the project runs Clippy in pedantic mode.

Dylint

Dylint is a linter for Rust developed by Trail of Bits. It can be installed by running the command `cargo install cargo-dylint dylint-link`. To run Dylint, we added a `Cargo.toml` file to the root of the repository with the following content.

```
[workspace.metadata.dylint]
libraries = [
  { git = "https://github.com/trailofbits/dylint", pattern = "examples/general/*" },
]
```

Figure C.1: Metadata required to run Dylint

To run the tool, run `cargo dylint --all --workspace`.

Semgrep

Semgrep can be installed using `pip` by running `python3 -m pip install semgrep`. To run Semgrep on a codebase, run `semgrep --config "<CONFIGURATION>"` in the root directory of the project. Here, `<CONFIGURATION>` can be a single rule, a directory of rules, or the name of a ruleset hosted on the Semgrep registry. Trail of Bits' public ruleset can be used by running `semgrep --config "p/trailofbits"`.

zizmor

`zizmor` can be installed using Cargo by running `cargo install zizmor --locked`. To analyze the GitHub workflows in the `Near-One/mpc` repository, navigate to the root directory of the repository and run `zizmor .`. To enable pedantic output, run the tool with the `-p` flag.

D. The Noise Protocol Framework

Noise is a lightweight framework for building encrypted transport protocols. It uses modern primitives, but unlike TLS, it does not rely on certificates or the web PKI.

The Noise Handshake

Noise uses an ECDH-based handshake to establish a shared secret. Depending on which handshake pattern is used, the handshake provides different security guarantees for the established connection. When both participants' static key pairs are known, it is possible to use the Noise KK[1] handshake, which provides strong security guarantees, on par with TLS 1.3. (For details, see section 7.7 of the Noise documentation.)

Implementing the Noise KK Handshake

The `mcginty/snow` repository contains a mature and maintained implementation of Noise for Rust. To set up an encrypted channel using Snow, first specify the Noise handshake pattern and convert it into a `Builder`, which is used to create the session initiator or responder as applicable.

```
// Set up the initiator and responder. Public and private keys are
// represented as 32-byte, fixed-size arrays.
let params: NoiseParams = "Noise_KK_25519_ChaChaPoly_BLAKE2s".parse()?;
let mut sender = Builder::new(params.clone())
    .prologue([sender_peer_id, receiver_peer_id].concat())])
    .local_private_key(&sender_private_key)?
    .remote_public_key(&receiver_public_key)?
    .build_initiator()?;
// The receiver looks up the public key of the sender based on the
// sender's peer ID.
let mut receiver = Builder::new(params)
    .prologue([sender_peer_id, receiver_peer_id].concat())])
    .local_private_key(&receiver_private_key)?
    .remote_public_key(&sender_public_key)?
    .build_responder()?;
```

Figure D.1: Set up a Noise KK channel using X25519 for the ECDH handshake, ChaCha20-Poly1305 for the encrypted transport, and BLAKE2s for the hash function

The `write_message` and `read_message` methods are used to encrypt and decrypt data from one participant to the other. Since we are using the Noise KK handshake, the handshake takes one round trip to complete.

[1] The first "K" means that the static key of the initiator is known to the responder, and the second "K" means that the static key of the responder is known to the sender.

```
// Define buffers to hold outgoing and incoming messages.
let mut ciphertext = [0u8; 200];
let mut plaintext = [0u8; 200];

let len = sender.write_message(b"", &mut ciphertext)?;
// Send the first handshake message over the network.
receiver.read_message(&ciphertext[..len], &mut plaintext)?;

let len = receiver.write_message(b"", &mut ciphertext)?;
// Send the second handshake message over the network.
sender.read_message(&ciphertext[..len], &mut plaintext)?;
```

Figure D.2: Completing the Noise KK handshake takes one full round trip. Here, we simply pass an empty byte string to the `write_message` method since we do not want to include any additional encrypted data as part of the handshake message.

The Noise KK handshake supports zero-round-trip encryption, and it is possible to start sending encrypted data as part of the handshake message. However, if this is not a strict requirement, we recommend that users do not include encrypted data as part of the handshake, as the security guarantees provided by the protocol for such data are weaker. (For details, see section 7.7 of the Noise documentation.)

When the handshake is complete, we can transition to transport mode and start sending encrypted data. This is done using the `into_transport_mode` method.

```
// Transition to transport mode.
let mut sender = sender.into_transport_mode()?;
let mut receiver = receiver.into_transport_mode()?;

let len = sender.write_message(b"This is a message", &mut ciphertext)?;
// Send the encrypted message over the network.
let len = receiver.read_message(&ciphertext[..len], &mut plaintext)?;

assert_eq!(&plaintext[..len], b"This is a message");
```

Figure D.3: When the handshake is complete, we transition to transport mode and start sending data.

Implementation Considerations

We note that even though Noise KK is conceptually simpler than TLS with dummy certificates, as is currently used by the networking layer, the identity misbinding issue TOB-NEARONE-6 could still have existed if the team had used Noise rather than TLS for the networking layer. It is important to ensure that the mapping from peer IDs to static keys is correct to ensure that the node knows who it is communicating with at all times.

To allow the receiver to determine the identity of the sender, we recommend that the sender sends their peer ID before the first handshake message. The peer IDs of the sender and receiver can be included in the handshake as part of the handshake prologue using the `Builder::prologue` method as follows.

```
// Set up the initiator and responder. Public and private keys are
// represented as 32-byte, fixed-size arrays.
let params: NoiseParams = "Noise_KK_25519_ChaChaPoly_BLAKE2s".parse()?;
let mut sender = Builder::new(params.clone())
    .prologue([sender_peer_id, receiver_peer_id].concat())])
    .local_private_key(&sender_private_key)?
    .remote_public_key(&receiver_public_key)?
    .build_initiator()?;
// The receiver looks up the public key of the sender based on the
// sender's peer ID.
let mut receiver = Builder::new(params)
    .prologue([sender_peer_id, receiver_peer_id].concat())])
    .local_private_key(&receiver_private_key)?
    .remote_public_key(&sender_public_key)?
    .build_responder()?;
```

Figure D.4: The peer IDs of the two participants can be included as part of the protocol prologue.

This provides cryptographic guarantees that both sender and receiver have the same view of the peer IDs of the two participants.

E. Considerations for Key Refresh and Resharing

Many modern MPC schemes involving digital signatures provide subprotocols known as key refresh and key resharing. These subprotocols allow all of the parties to rotate or refresh their secret signing key shares; key refresh keeps the same group of participants, whereas key resharing allows a new group of participants (and possibly threshold) to be created. These protocols are powerful because the newly generated shares are completely disconnected from their original shares. As a result, a compromise of a previous share does not allow an attacker to compromise newer shares. This allows the group signing key to remain the same, so normal signing functionality is not disrupted.

It is considered a best practice to integrate key resharing into MPC applications and to perform regular key refreshes after a predetermined period of time. If key refresh is automated, it reduces the window of compromise of secret shares in the system, and it makes it more difficult for an attacker to exploit the system if they are able to compromise any party's secret share.

However, care is required to ensure that the key resharing protocol cannot be compromised to either expose the private key or corrupt the key shares so that signing is no longer possible. In particular, we recommend that the NEAR One team consider doing the following:

- Key refresh is the simpler of the two protocols to use because it does not involve changing the group's participants. Exercise caution when using key resharing. In particular, ensure that the new group has a safe group size and threshold. Also, ensure that members of the old group do not maintain access to their key shares.
- When using either key resharing or key refresh, ensure that the refresh completes successfully before deleting the original key shares. In other words, there should be validation to ensure that no errors occurred during refresh and that all parties have successfully generated new secret shares before the old secret shares are deleted. If the original key share is deleted and the refresh did not complete successfully, this would be catastrophic to the system, as the MPC nodes will no longer have the ability to generate signatures. We strongly recommend adding positive and negative tests for this property to ensure that this problem does not arise. In particular, add multiple tests to ensure that old key shares can still perform signing operations when an error occurs.
- Once you have confirmed that resharing has completed successfully, you should securely delete the original key share to complete the key rotation and ensure that old shares cannot be compromised. Like the previous point, we recommend implementing extensive positive and negative tests for this property; in particular, the following properties should be tested for:

1. The new group satisfies the expected parameters in terms of group size and threshold.
2. The new key shares can be used to sign messages, and the new group public key is equal to the previous group public key.
3. Old key shares are successfully restored if the resharing protocol fails.
4. Old key shares are deleted and cannot be used for further operations if the resharing protocol completes successfully.

- Once key refresh is implemented, set a policy for automatically refreshing keys after some predetermined amount of time or number of transactions. This will ensure that the window of compromise for any key share is restricted to the window set by the configured policy.

References

Section 3 of the paper "Attacking Threshold Wallets" by Aumasson and Shlomovits describes a key-destruction attack targeting a common key resharing protocol. The blog post "Breaking the shared key in threshold signature schemes" by Trail of Bits describes another common vulnerability in resharing protocols based on Feldman verifiable secret sharing.

- Attacking Threshold Wallets
- Breaking the shared key in threshold signature schemes

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at https://github.com/trailofbits/publications, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review projects, supporting client organizations in the technology, defense, and finance industries and government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, and Zoom.

Trail of Bits also operates a center of excellence with regard to blockchain security. Notable projects include audits of Algorand, Bitcoin SV, Chainlink, Compound, Ethereum 2.0, MakerDAO, Matic, Uniswap, Web3, and Zcash.

To keep up to date with our latest news and announcements, please follow @trailofbits on Twitter and explore our public repositories at https://github.com/trailofbits. To engage us directly, visit our "Contact" page at https://www.trailofbits.com/contact or email us at info@trailofbits.com.

Trail of Bits, Inc.
228 Park Ave S #80688
New York, NY 10003
https://www.trailofbits.com
info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2025 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to NEAR One under the terms of the project statement of work and has been made public at NEAR One's request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the Trail of Bits Publications page. Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.